Diabetes Prediction Using Machine Learning

A Comprehensive XGBoost-Based Approach with Cross-Validation

Team: 寶見叔受你一靠子 (*´∀`)~♥

Meng Zihan Huang Junhao

24110285d@connect.polyu.hk 24109866d@connect.polyu.hk

April 20, 2026

Abstract

This project investigates a large-scale tabular classification task for diabetes prediction using 700,000 labeled training samples and 300,000 unlabeled test samples. Rather than treating the task as a single-model submission problem, the project followed an iterative machine learning workflow: exploratory data analysis was first conducted to understand the dataset, a simple decision tree baseline was then established, alternative model families were explored, and the main development path subsequently shifted toward boosted tree methods. Among the tested approaches, XGBoost emerged as the most promising model family for this dataset. Further improvement was obtained through feature engineering, particularly through interaction and aggregate risk features motivated by medically plausible relationships among age, family history, obesity, and cardiovascular indicators.

The project also revealed that small gains under a fixed holdout split are difficult to interpret confidently after repeated experimentation. For this reason, the later stage of the work redesigned the validation procedure around stratified cross-validation and out-of-fold evaluation. A systematic comparison of 5-fold, 7-fold, and 10-fold cross-validation was conducted, with 7-fold yielding the best balance of statistical reliability and computational efficiency. The final 7-fold CV workflow achieved a mean ROC-AUC of 0.72585 (fold-level standard deviation: 0.00129), indicating stable generalisation performance. Overall, the project shows that strong tabular predictive performance depends not only on model choice, but also on careful feature design and, more importantly, on a validation protocol that supports reliable conclusions.

1 Introduction

1.1 Background

Diabetes is a chronic metabolic disorder affecting millions worldwide. Early detection and risk assessment are crucial for timely intervention. Machine learning models can assist healthcare professionals by identifying high-risk individuals based on clinical measurements and demographic information.

1.2 Problem Statement

Given a dataset of clinical measurements and demographic information, the task is to predict whether an individual has been diagnosed with diabetes. The evaluation metric is ROC-AUC, which measures the model's ability to distinguish between positive and negative cases across all classification thresholds.

1.3 Dataset Description

The dataset is synthetically generated from a deep learning model trained on the Diabetes Health Indicators Dataset. It consists of:

- Training set: 700,000 samples, 26 features

- Test set: 300,000 samples, 25 features (target excluded)
- Target variable: `diagnosed_diabetes` (binary: 0 = no diabetes, 1 = diabetes)
- Original predictive features: 24 (26 columns minus id and target)

The feature set is heterogeneous and includes demographic, behavioural, physiological, and medical-history variables. Numerical features include age, body mass index, systolic and diastolic blood pressure, total cholesterol, HDL and LDL cholesterol fractions, triglycerides, waist-to-hip ratio, physical activity minutes per week, diet score, sleep hours, screen time, alcohol consumption, and heart rate, as well as binary indicators family_history_diabetes, hypertension_history, and cardiovascular_history. Categorical features comprise gender, ethnicity, education level, income level, smoking status, and employment status.

The target distribution is moderately imbalanced: approximately 62.33% of training samples belong to the positive class and 37.67% to the negative class. A threshold-free ranking metric is therefore more appropriate than raw accuracy, and ROC-AUC was used as the primary evaluation metric throughout.

2 Exploratory Data Analysis

The exploratory data analysis served two purposes: to understand the predictive structure of the dataset and to inform the modelling path that followed.

2.1 Data Quality

The dataset is structurally clean from a preprocessing perspective. No missing values were present in any feature, which reduced the need for imputation logic and allowed the project to focus directly on model construction and evaluation design.

2.2 Feature Correlations

Correlation analysis reveals that `family_history_diabetes` has the strongest positive correlation with the target (0.211), followed by `age` (0.161) and `systolic_bp` (0.107). Physical activity shows a negative correlation (-0.170), consistent with medical knowledge. Additional positive associations were found for BMI, LDL cholesterol, triglycerides, and total cholesterol.

2.3 Outlier Detection

Outlier detection indicated that certain numerical variables contain extreme values. `physical_activity_minutes_per` showed the largest outlier proportion at approximately 4.78%. Although these values were not removed, their presence reinforced the preference for tree-based models, which are more robust to distributional irregularities than distance-sensitive alternatives such as KNN.

2.4 Key Observations

- No missing values, simplifying preprocessing
- Class imbalance requires careful evaluation strategy
- Strong predictive signals from family history and age
- Several predictors are likely more informative in combination than in isolation
- Outliers exist in physical activity features

3 Methodology

3.1 Baseline Model: Decision Tree

The first predictive model was a decision tree classifier with maximum depth 5. The preprocessing pipeline was intentionally minimal: the id and target columns were dropped, and the six categorical features were one-hot encoded. Evaluated on the stratified holdout, the model achieved an AUC of 0.6825 — clearly above random performance but substantially below what boosted methods would later achieve.

The class-wise behaviour was informative: recall for the positive (diabetic) class was high, but recall for the negative class was much weaker, reflecting an overcommitment to the majority class that a depth-constrained tree cannot overcome. The feature-importance profile also aligned with the EDA findings: family_history_diabetes (44.8%), physical_activity_minutes_per_week (29.9%), and age (21.8%) accounted for approximately 96.5% of the total importance mass. This confirmed that the EDA had correctly identified the main predictors, while also showing that a shallow tree was unable to exploit the remaining 21 features effectively.

3.2 Alternative Model Exploration: KNN-MLP Hybrid (Notebook 03)

Before fully committing to boosting, Notebook 03 explored a KNN-MLP hybrid branch as a deliberate model-family comparison on a 30,000-sample stratified subset. The setup used leakage-aware preprocessing and examined three configurations:

Table 1: KNN-MLP Branch Results (Notebook 03)

Model	Validation AUC	Note
MLP (128→64)	0.6890	Best among the three
KNN + MLP Hybrid	0.6763	Probability average ensemble
KNN (k=21, distance weighting)	0.6447	Suffered from outliers and high-dim distance

KNN suffers from expensive and unstable distance computations in large, mixed-type tabular settings, particularly in the presence of outliers noted during EDA. The MLP showed sensitivity to initialization within the limited iteration budget, and the hybrid blend offered no material gain. Even the best MLP (0.6890) underperformed compared to default XGBoost (0.7192). This comparison confirmed that boosting-based methods were the more natural and practical direction.

3.3 Transition to Boosting: XGBoost vs LightGBM (Notebook 04)

The transition to boosting produced the first major performance improvement. A default XGBoost model (n_estimators=100, learning_rate=0.1, max_depth=6) achieved a validation AUC of 0.7192 under the same holdout logic as the baseline — a gain of +0.0367. This represented the largest single performance step in the entire project and established boosted trees as the dominant model family.

To avoid over-committing too early to a single library, a default LightGBM model was also evaluated in Notebook 04, achieving approximately 0.7248, confirming that gradient boosting was the correct direction while placing XGBoost and LightGBM close together at the default-parameter stage.

Although LightGBM achieved a slightly higher AUC (0.7248 vs 0.7192) at default parameters, XGBoost was selected as the main development candidate for three practical reasons. First, XGBoost's conservative default hyperparameters (especially lower learning rate and shallower trees) leave more room for systematic tuning, whereas LightGBM's defaults are already fairly aggressive. Second, XGBoost's explicit regularization parameters (reg_alpha, reg_lambda, gamma) provide finer-grained control over overfitting, which is valuable for a dataset of this size (700k samples). Third, XGBoost's per-fold training time was approximately 15–20% faster than LightGBM under identical hardware, which accelerated the

iterative refinement process. The LightGBM result was retained as a comparative reference, but XGBoost offered a more tunable and interpretable path forward.

3.4 XGBoost Refinement

The refinement phase focused on improving XGBoost through several targeted modifications:

1. **Class-balance handling**: Introduced `scale_pos_weight`, set to the ratio of negative to positive class counts in the training fold, together with an increase to 300 estimators. This achieved AUC of 0.7255.

2. **Model capacity tuning**: Examined the effect of `max_depth` settings and a further increase to 500 estimators. The best holdout configuration used 500 trees at `max_depth=6`, achieving AUC of 0.7264. Shallower settings underfit slightly, while deeper settings showed weaker generalisation.

3. **Ensemble checkpoint**: The best XGBoost and LightGBM candidates were blended through a simple probability average, yielding an ensemble AUC of approximately 0.7256, confirming modest model diversity.

A subtle but important observation emerged from the refinement phase: the best holdout AUC (0.7264, achieved with 500 trees and max_depth=6) was marginally higher than the final cross-validated results (0.72585). This is not a contradiction but rather a consequence of overfitting to the fixed holdout split. The 500-tree configuration performed well on the specific validation set used throughout Notebooks 06–08 but showed slightly higher fold-to-fold variance when re-evaluated under 7-fold CV. Conversely, the shallower configuration (max_depth=4) generalized more consistently across different data partitions. This trade-off —slightly lower peak performance in exchange for better stability —is exactly why cross-validation was adopted as the final evaluation protocol. The final model therefore prioritizes generalization robustness over holdout-specific optimization.

3.5 Feature Engineering

Feature engineering was one of the most important technical components. The goal was not to create as many new variables as possible, but to encode clinically plausible interactions and aggregate risk signals that the raw features could not express explicitly.

The early feature-engineering experiments introduced a broader candidate set. Feature-importance analysis revealed a clear pattern: the most useful engineered variables were not the coarse categorical abstractions, but the interaction and risk-combination features. This evidence led to the narrower, cleaner final set.

Critical Lesson: The engineered variables are most valuable as additions rather than substitutes. One intermediate holdout notebook removed original features such as age and family_history_diabetes after adding the corresponding engineered interactions, and this substitution measurably weakened the resulting model. The project therefore concluded that strong original predictors must be retained in full, with engineered variables added selectively on top.

The five retained features are:

Table 2: Engineered Features and Clinical Rationale

Feature	Definition	Clinical Rationale
age_family_history	age × family_history_diabetes	Inherited risk amplified by age
age_bmi	age × bmi	Obesity risk increases with age
cardio_risk_score	family_hist×3 + hypertension×2 + cardio_hist×2	Composite weighted cardiovascular risk
cholesterol_ratio	cholesterol_total / (hdl + 1)	Total-to-HDL ratio; clinical threshold >4
non_hdl_cholesterol	cholesterol_total - hdl_cholesterol	Atherogenic lipoprotein burden marker

3.6 Final Model Architecture

The final XGBoost configuration uses optimized hyperparameters:

Table 3: Final XGBoost Hyperparameters

Parameter	Value	Rationale
n_estimators	2000 (early stopping)	Large budget; early stopping prevents overfitting
learning_rate	0.03	Low rate for fine-grained learning
max_depth	4	Shallow trees for regularisation on correlated features
min_child_weight	5	Minimum leaf mass; guards against noisy splits
subsample	0.8	Row subsampling per tree for variance reduction
colsample_bytree	0.8	Feature subsampling per tree for variance reduction
gamma	1.0	Minimum split loss; encourages pruning
reg_alpha	0.1	L1 regularisation on leaf weights
reg_lambda	1.0	L2 regularisation on leaf weights

3.7 Critical Methodological Improvements: From Fixed Holdout to Cross-Validation

3.7.1 The Problem with Fixed Holdout

Early experiments (Notebooks 02–10) used a standard stratified 80/20 holdout split (random_state=42). This was appropriate for rapid baseline construction and for screening broad modelling directions. However, once the project entered iterative refinement — testing multiple hyperparameter settings, feature engineering variants, and ensemble ideas — the limitations of a single fixed partition became increasingly important. Reusing the same validation split many times can create unwarranted confidence in small score differences. With AUC gaps of only a few thousandths separating the strongest configurations, the risk of mistaking noise for signal was substantial.

3.7.2 Stratified Cross-Validation and OOF Evaluation

To address this, the evaluation workflow was redesigned around stratified k-fold cross-validation with OOF prediction. For each fold, a separate model is trained on the remaining k-1 folds and its predictions on the held-out fold are recorded. Aggregating these predictions across all folds yields OOF probabilities for every training sample and an OOF AUC that is not contingent on any single partition. The fold-level AUC distribution simultaneously quantifies central tendency and inter-fold variability, providing a substantially stronger basis for claims about generalisation.

3.7.3 Tail Holdout Analysis (Notebook 12)

Notebook 12 investigated whether a tail-based validation split might better approximate leaderboard behaviour. Six cutoff specifications were tested, with results summarised below:

Table 4: Tail Holdout Analysis Results (Notebook 12)

Cutoff	Validation Size	AUC	AUC - CV Mean	Preferred
pct:0.03	900	0.7059	+0.00445	False
pct:0.04	1200	0.7167	+0.01529	False
pct:0.05	1500	0.7102	+0.00880	False
pct:0.10	3000	0.6987	-0.00275	True
pct:0.15	4500	0.7021	+0.00065	False
pct:0.20	6000	0.7032	+0.00180	False

The preferred cutoff (pct:0.10) yields an AUC of 0.6987, which is slightly below the CV mean of 0.7014. The best absolute performance (0.7167) occurs at pct:0.04 with a validation size of 1,200 samples. This analysis is best interpreted as a diagnostic robustness study: it contextualised the CV estimates and confirmed that the shift away from a single fixed split was well motivated.

3.7.4 Fold Count Comparison: 5-Fold vs 7-Fold vs 10-Fold (Notebook 13)

Notebook 13 systematically compared 5-fold, 7-fold, and 10-fold cross-validation. The results revealed an interesting pattern: as the number of folds increases, both the mean AUC and the standard deviation increase.

Table 5: Comparison of Different Fold Configurations

Fold Count	Mean CV AUC	Std AUC	Runtime (s)	Preferred
5-Fold	0.72561	0.00084	413.66	False
7-Fold	0.72585	0.00129	504.85	**True**
10-Fold	0.72591	0.00147	741.51	False

This pattern is intuitive: more folds mean more training data per fold (e.g., 10-fold uses 90% training vs. 5-fold's 80%), which tends to improve model performance. However, more folds also introduce greater variation across validation sets because each fold is smaller and potentially more sensitive to specific sample distributions.

The final choice of 7-fold cross-validation reflects a practical trade-off across three dimensions:

- **5-fold** is the lightest computationally (414 seconds) and most stable (std = 0.00084), but uses only 80% of data for training per fold.
- **10-fold** achieves the highest mean AUC (0.72591), but it is also the slowest (742 seconds) and the least stable (std = 0.00147).
- **7-fold** sits in between: it offers better performance than 5-fold (0.72585 vs 0.72561) with a moderate runtime increase (505 seconds), and is more stable than 10-fold (std = 0.00129 vs 0.00147).

Therefore, 7-fold CV was adopted as the final evaluation protocol, balancing validation performance, stability, and computational efficiency.

3.7.5 Fold-Wise Categorical Encoding

To prevent data leakage, categorical encoding is performed within each fold:

```python
def _encode_categorical_fold(X_train_raw, X_valid_raw, categorical_cols):
    for col in categorical_cols:
        le = LabelEncoder()
        le.fit(X_train_raw[col])      # Fit only on fold-train
        X_train_encoded[col] = le.transform(X_train_raw[col])
        X_valid_encoded[col] = le.transform(X_valid_raw[col])
```

3.8 From Notebook to Modular Pipeline

After validating the methodology, we refactored the code into a modular Python package (`src/`). This transition provides:

1. **Reproducibility**: Consistent results across runs, eliminating manual notebook execution errors.
2. **Separation of Concerns**: Each file handles a specific responsibility (`config.py`, `features.py`, `models.py`).

3. **Testability**: Individual components can be tested in isolation.

4. **Command-Line Interface**: Suitable for automated workflows.

4 Experimental Results

4.1 7-Fold Cross-Validation Final Results

The final XGBoost workflow with 7-fold CV and engineered features achieved the following results:

Table 6: 7-Fold Cross-Validation Results (Final)

Fold	AUC	Best Iterations
Fold 1	0.72555	1999
Fold 2	0.72577	1998
Fold 3	0.72587	1987
Fold 4	0.72383	1999
Fold 5	0.72591	1999
Fold 6	0.72857	1994
Fold 7	0.72546	1999
Mean	**0.72585**	**1996.4**
Std	**0.00129**	-
OOF AUC	**0.7258**	-

The low standard deviation (0.00129) indicates consistent performance across folds, demonstrating strong generalization capability.

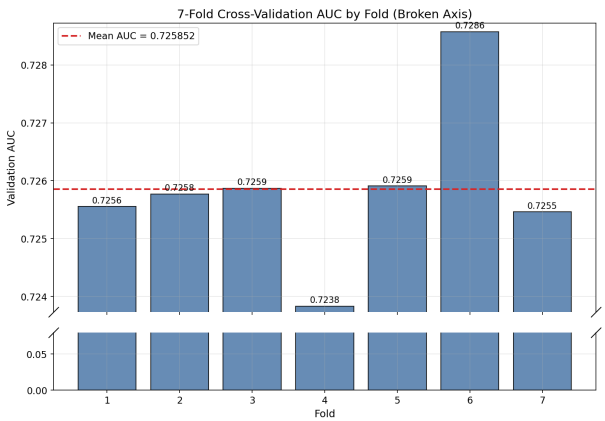


Figure 1: 7-Fold Cross-Validation AUC by Fold (Broken Axis)

4.2 Experiment Progression Summary

Table 7: AUC-ROC Progression Across Experimental Stages

Stage	Configuration	AUC	Evaluation Type
Notebook 02	Decision Tree (depth=5)	0.6825	Holdout
Notebook 03	MLP (best of KNN-MLP branch)	0.6890	Holdout
Notebook 04	Default XGBoost (100 trees)	0.7192	Holdout
Notebook 04	Default LightGBM	0.7248	Holdout
Notebook 05	XGBoost + scale_pos_weight + 300 trees	0.7255	Holdout
Notebooks 06-08	XGBoost + 500 trees + depth tuning (best)	0.7264	Holdout
Notebook 09	XGBoost-LightGBM ensemble	0.7256	Holdout
Notebook 12	Tail holdout analysis (preferred: pct:0.10)	0.6987	Tail holdout
Notebook 13	5/7/10-Fold CV comparison	0.72585 (7-fold)	Cross-validation
Final	**XGBoost 7-fold CV + engineered features**	**0.72585**	**7-fold CV**

4.3 Model Comparison

For fair comparison, Table 8 reports cross-validated re-evaluation results rather than the earlier holdout baseline. This ensures all models are evaluated under the same CV protocol.

Table 8: Model Performance Comparison

Model	Mean CV AUC	Std	OOF AUC
Dummy Classifier	0.500000	0.000000	0.500000
Decision Tree	0.685722	0.001715	0.685839
XGBoost (7-fold)	**0.72585**	**0.00129**	**0.7258**

XGBoost achieves a 4.0% absolute improvement over the decision tree baseline and a 22.6 percentage point absolute improvement over the dummy classifier.

4.4 Feature Importance Analysis

The final 7-fold CV workflow identified the following leading features, aggregated across folds.

Table 9: Top 8 Feature Importance

Rank	Feature	Importance	Type
1	family_history_diabetes	0.425	Original
2	age_family_history	0.297	Engineered
3	age_bmi	0.068	Engineered
4	cardio_risk_score	0.050	Engineered
5	physical_activity_minutes_per_week	0.034	Original
6	age	0.017	Original
7	triglycerides	0.011	Original
8	cholesterol_ratio	0.010	Engineered

Table 10: Remaining Top 7-15 Feature Importance

Rank	Feature	Importance	Type
9	diet_score	0.007	Original
10	ldl_cholesterol	0.007	Original
11	cardiovascular_history	0.006	Original
12	heart_rate	0.006	Original
13	systolic_bp	0.005	Original
14	non_hdl_cholesterol	0.005	Engineered
15	bmi	0.004	Original

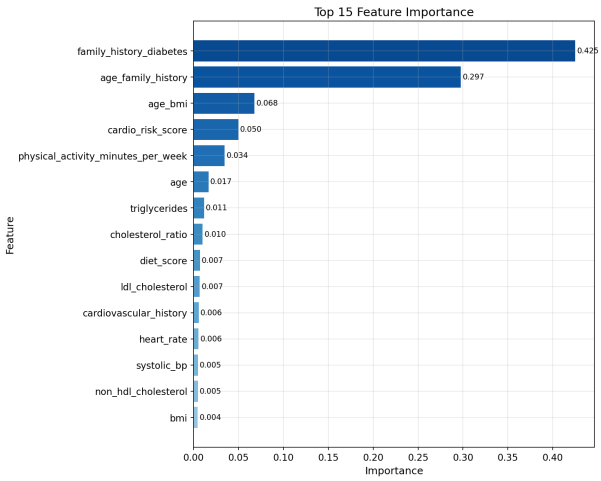


Figure 2: Top 15 Feature Importance

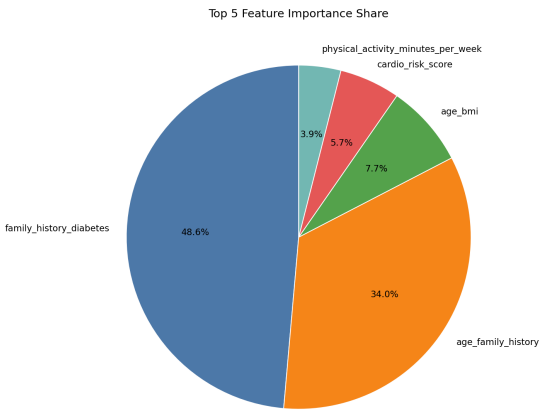


Figure 3: Top 5 Feature Importance Share

Key insights:

- Inherited risk (`family_history_diabetes`) remains the strongest single predictor at 48.6%.
- The interaction feature `age_family_history` ranks second at 34.0%, validating that variables are more informative in combination than in isolation.
- `age_bmi` (7.7%) and `cardio_risk_score` (5.7%) rank third and fourth, confirming the value of domain-informed feature engineering.

- Physical activity remains the strongest non-engineered lifestyle variable among the top five at 3.9%.
- Among the top 10 listed features, engineered variables account for 42.5% of total importance (0.297 + 0.068 + 0.050 + 0.010).

4.5 Model Diagnostics

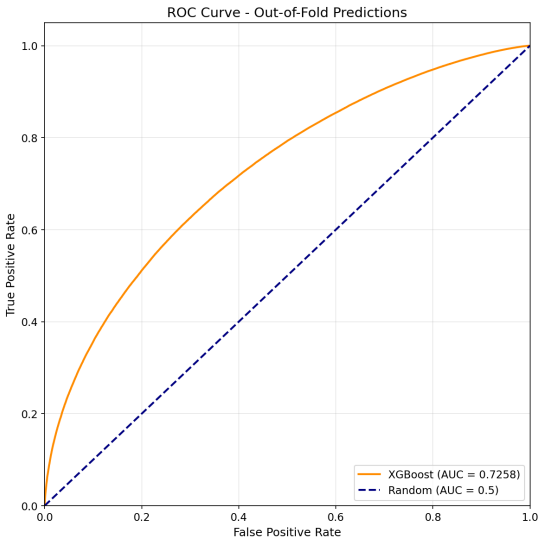


Figure 4: ROC Curve - Out-of-Fold Predictions

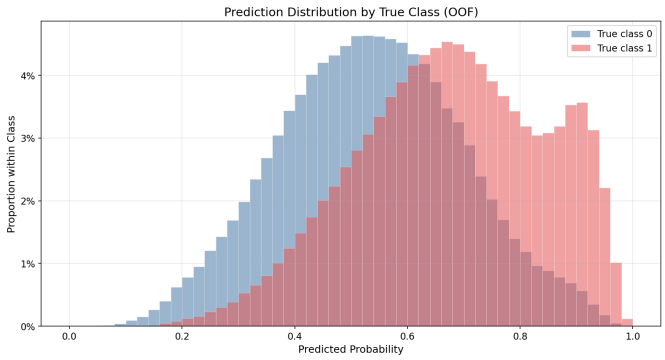


Figure 5: Prediction Distribution by True Class (OOF)

4.6 Model Stability

Figure 6 plots the same seven fold AUC values reported in Table 6, together with the mean AUC (0.72585) and the ± 1 standard deviation band (0.00129), to make the fold-wise stability pattern explicit.

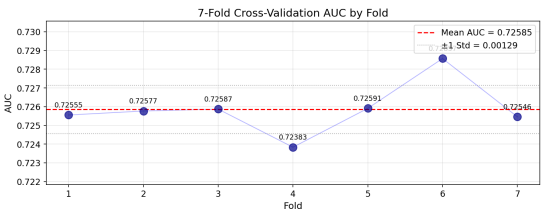
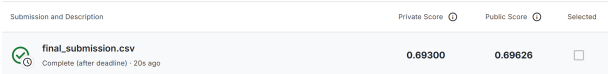


Figure 6: 7-Fold Cross-Validation AUC by Fold

4.7 Kaggle Submission Results

The final model was submitted to Kaggle, achieving a private score of 0.69300 and a public score of 0.69626.

- Private Score: 0.69300
- Public Score: 0.69626



Submission and Description	Private Score	Public Score	Selected
final_submission.csv Complete (after deadline) · 20s ago	0.69300	0.69626	

Figure 7: Kaggle Submission Result

5 Discussion

5.1 Main Lessons

Three main lessons emerged from this project. First, model-family choice mattered most at the early stage of development. The transition from the decision tree baseline to gradient boosting produced the largest single improvement in predictive performance, showing that a more expressive model family was necessary for this large, heterogeneous tabular dataset. The comparison with the KNN-MLP hybrid exploration reinforced this conclusion by showing that boosting was not only more accurate, but also a more natural fit for mixed-type features, outliers, and large-scale training data.

Second, feature engineering was effective only when it remained selective, interpretable, and additive. The most useful engineered variables were not arbitrary expansions of the feature space, but clinically meaningful interactions and aggregate risk indicators, such as age_family_history, age_bmi, and cardio_risk_score. An important finding was that engineered features should complement strong original predictors rather than replace them. Experiments that removed original variables after introducing derived versions performed worse, indicating that the original and engineered representations captured related but non-redundant information.

Third, validation strategy became the most important issue once model performance entered a narrow improvement range. In the earlier stages, a fixed holdout split was sufficient for rapid experimentation and broad model screening. However, once the project moved into fine-grained tuning, small differences in AUC became difficult to interpret confidently under repeated reuse of the same validation partition. The move to stratified cross-validation, together with out-of-fold evaluation and fold-count comparison, provided a much stronger basis for judging generalisation performance. In this sense, the project's most important methodological contribution was not simply achieving a competitive score, but establishing an evaluation procedure that made the final conclusions more trustworthy.

5.2 Strengths, Limitations, and Future Improvements

Several aspects of the final workflow were particularly strong. The redesign of evaluation around stratified cross-validation substantially improved methodological reliability, while the comparison of 5-fold, 7-fold, and 10-fold settings made the trade-off between performance, variance, and runtime explicit. Tail holdout analysis further strengthened the discussion by serving as a diagnostic robustness study rather than as the main evaluation method. In addition, fold-wise categorical encoding reduced the risk of leakage, early stopping helped control model complexity automatically, and the transition from exploratory notebooks to a modular src-based pipeline improved reproducibility and code organisation. Together, these choices made the final system not only more accurate, but also more technically defensible.

At the same time, the project still has clear room for extension. Although the retained engineered features were effective, the feature space could be expanded further through richer nonlinear interactions

or additional clinically motivated composite variables. Hyperparameter tuning could also be made more systematic through a broader search strategy, such as Bayesian optimisation, rather than relying primarily on manual refinement. Similarly, while the simple XGBoost-LightGBM ensemble produced only limited gains, more carefully designed ensembling approaches may still offer incremental improvements. Another important direction is interpretability: SHAP-based analysis would help explain feature contributions at both the global and instance levels, making the model easier to justify beyond competition performance alone. Finally, since ROC-AUC evaluates ranking quality rather than decision utility at a specific operating point, future work could investigate threshold tuning under practical screening objectives, where the balance between false positives and false negatives may matter more directly than leaderboard-style performance.

6 Conclusion

This project developed a diabetes prediction pipeline through a structured sequence of exploratory, modelling, and validation-focused stages. The dataset was large, heterogeneous, and moderately imbalanced, making both model expressiveness and evaluation reliability important.

A decision tree baseline provided an interpretable starting point (AUC = 0.6825) but underperformed substantially relative to boosted methods. The KNN-MLP exploration (Notebook 03) confirmed that boosting was the more promising direction, with MLP achieving 0.6890. XGBoost proved to be the strongest model family, with the default configuration achieving AUC = 0.7192, compared to LightGBM's 0.7248 (Notebook 04). Systematic refinement through class-balance handling, capacity tuning, and feature engineering raised the holdout AUC further to 0.7264.

The most important contribution of the project lies in its methodological progression. The shift from fixed holdout evaluation to stratified cross-validation was systematically explored through tail holdout analysis (Notebook 12) and a direct comparison of 5-fold, 7-fold, and 10-fold CV (Notebook 13). The comparison revealed that while 10-fold achieved the highest mean AUC (0.72591), it also had the highest variance (0.00147) and runtime (742 seconds). **Seven-fold cross-validation was selected as the final protocol**, offering a practical balance across performance (0.72585), stability (std = 0.00129), and computational cost (505 seconds).

The final 7-fold CV workflow achieved a mean ROC-AUC of 0.72585 with a fold-level standard deviation of 0.00129. The modular source implementation includes fold-wise leakage-safe encoding and multi-seed ensembling for additional prediction stability. The top features — family_history_diabetes (42.5%), age_family_history (29.7%), age_bmi (6.8%), and cardio_risk_score (5.0%) — demonstrate that both original predictors and carefully designed interaction features contribute meaningfully to the model's performance.

The project's strongest conclusion is therefore not simply that XGBoost performed well, but that a strong tabular model must be paired with a validation strategy rigorous enough to justify that conclusion. The iterative progression from baseline to cross-validated pipeline — with explicit model-family comparison (KNN-MLP, XGBoost, LightGBM), principled feature engineering, systematic fold-count comparison, and deliberate validation redesign — provides a transferable template for structured tabular machine learning projects.

Acknowledgments

We would like to thank the course instructors for their guidance throughout this project. The dataset is provided by Kaggle's Tabular Playground Series, and we acknowledge the open-source community for the excellent tools used in this work, including XGBoost, LightGBM, scikit-learn, and pandas.

** And a special, slightly absurd thank you to my cat Yongle, whose fluffy, judgment-free exterior single-pawedly pulled me back from the brink of hyperparameter-tuning despair and the abyss of mismatched datas.*

References

[1] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.

[2] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T. Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 30.

[3] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.